

Backend Developer — Step 4

REST APIs

Roadmap objective: Design and implement resource-oriented HTTP APIs

Level: Beginner → Intermediate

Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- HTTP methods
- URLs/resources
- Status codes
- JSON
- REST constraints
- CRUD mapping
- Validation
- Pagination/filtering
- API testing

Core concepts

1. A REST-style API exposes resources through predictable URLs and uses HTTP semantics consistently.
2. GET should retrieve, POST commonly creates, PUT replaces, PATCH partially updates, and DELETE removes.
3. Status codes communicate outcome; do not return 200 for every failure.
4. API contracts should be documented and tested independently of the frontend.

Concept map

```
REST APIs
|
+--> Learn the concept
|
+--> Reproduce a small example
|
+--> Apply it in a feature
|
+--> Test failure/edge cases
|
+--> Document the decision
```

Detailed study guide

1. HTTP methods

Study http methods through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. URLs/resources

Study urls/resources through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. Status codes

Study status codes through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. JSON

Study json through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. REST constraints

Study rest constraints through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. CRUD mapping

Study crud mapping through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

REST route shape

```
GET    /api/users
GET    /api/users/42
POST   /api/users
PATCH /api/users/42
DELETE /api/users/42
```

```
A successful GET might return:
200 OK
Content-Type: application/json
```

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
GET	Read	Should be safe/idempotent in normal REST usage
POST	Create/action	Often non-idempotent
PUT	Replace	Idempotent when designed so
PATCH	Partial update	Changes selected fields
DELETE	Remove	Deletes resource

Common mistakes

- Using the wrong HTTP method
- Returning 200 for every outcome
- Unclear URLs
- No validation
- No API documentation/testing

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Design five resource URLs before writing code.
- Write down what you expected, what happened, and why.

Lab 2

- Implement GET and POST first.
- Write down what you expected, what happened, and why.

Lab 3

- Add PATCH and DELETE.
- Write down what you expected, what happened, and why.

Lab 4

- Return meaningful 2xx/4xx status codes.
- Write down what you expected, what happened, and why.

Lab 5

- Add validation for malformed input.
- Write down what you expected, what happened, and why.

Lab 6

- Test every endpoint in Postman and record expected responses.
- Write down what you expected, what happened, and why.

Mini-project

Build a REST Task API with CRUD endpoints, validation, pagination and correct status codes. Test every endpoint with Postman.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

GET vs POST?

- Answer structure: definition → example → trade-off/limitation → practical use.

PUT vs PATCH?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is a 404?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is a 400?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why validate on the server?

- Answer structure: definition → example → trade-off/limitation → practical use.

How would you paginate an endpoint?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
HTTP	Write this in your own words.	Give one project example.
Endpoint	Write this in your own words.	Give one project example.
Resource	Write this in your own words.	Give one project example.
Method	Write this in your own words.	Give one project example.
Status code	Write this in your own words.	Give one project example.
JSON	Write this in your own words.	Give one project example.
Idempotent	Write this in your own words.	Give one project example.
Validation	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Pagination	Write this in your own words.	Give one project example.
API contract	Write this in your own words.	Give one project example.

Self-test

- Explain HTTP methods without using its textbook definition.
- Show a small example of URLs/resources.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate HTTP methods.
- I can explain and demonstrate URLs/resources.
- I can explain and demonstrate Status codes.
- I can explain and demonstrate JSON.
- I can explain and demonstrate REST constraints.
- I can explain and demonstrate CRUD mapping.
- I can explain and demonstrate Validation.
- I can explain and demonstrate Pagination/filtering.
- I can explain and demonstrate API testing.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- <https://developer.mozilla.org/en-US/docs/Web/HTTP>
- <https://learning.postman.com/>

Recommended videos

- REST API Tutorial (Existing DB resource 106)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.